# claude-windows / 50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a\subagents\workflows\wf_ddb0b3fd-b40\agent-a5dc87f1a037c8089.jsonl`
- SHA-256: `a28148f919f5a093a5ee098b3344cea77d839ce2c218eb1f66b43d15da0745aa`
- Source modified: `2026-07-07T04:12:26+00:00`
- Imported at: `2026-07-08T16:00:26+00:00`
- project: `wf_ddb0b3fd-b40`
- session_id: `50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a`

## Transcript

### 1. user (2026-07-07T04:10:28.573Z)

You are reviewing an UNCOMMITTED working diff implementing issue #506 (GPU-resident WASB inference) in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. The full diff is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch (read it). The new test file tests/test_wasb_nvdec_resident.py is untracked (not in the diff) ? read it directly from the repo.

Ground truth you must compare against:
- The VALIDATED reference implementation: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/gpu_resident/exp_t2_full.py (the GpuFrontend class + its grid build ARE what was validated on a real L4: F1 0.574 ? baseline 0.582 on the hard clip). The production port must be semantically faithful to it.
- WASB-SBDT interfaces (not in this repo; trust these facts): TracknetV2Detector.run_tensor(imgs, affine_mats) moves imgs .to('cuda') itself and calls postprocessor.run(preds, affine_mats) which does affine_mats[scale].cpu().numpy() then indexes [i] per batch item; ImageDataset (eval) returns (imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths) where trans_outputs_inv is {scale: np.float64 [2,3]}; per-frame preprocessing is PIL RGB -> cv2.warpAffine(trans_input, (512,288), INTER_LINEAR) -> ToTensor -> Normalize(ImageNet); get_transform uses c=(w/2,h/2), s=max(h,w); OnlineTracker uses np.Inf; WASB detector.py calls torch.load(model_path) with no weights_only arg.
- The CPU decode path (run(), run_video_streaming) must be BEHAVIOR-UNCHANGED on the WSL torch-1.11 + numpy<1.24 env (byte-parity guarantee), except that torch>=2.6 envs get the weights_only + np.Inf shims.

Report ONLY defects you can ground in the actual code you read ? cite file:line. No style nits, no speculative maybes without a concrete failure scenario. Return findings as structured output.

A reviewer claims this defect in the #506 diff:
- file: backend/pipeline/detectors/wasb_infer.py
- line: 1298
- summary: main() gives --decode-backend nvdec_resident silent precedence over --stream-video and --frames_out_dir instead of rejecting the contradictory flag combination.
- failure scenario: An operator A/B-benchmarking the backends adds --stream-video to a command line that already carries --decode-backend nvdec_resident (e.g. copied from a runner debug log); the run silently executes the NVDEC path (returning at line 1320 before the stream_video branch), producing trajectory-level-different output under an nvdec-keyed cache with no warning ? the intended streaming-path measurement is silently invalid.
- severity: minor

Adversarially VERIFY it: read the actual code (and the validated reference / WASB interface facts above) and try to REFUTE the claim. It is only real if the concrete failure scenario actually happens in the code as written. Default to isReal=false if the scenario cannot occur, is already guarded, or rests on a misreading.

**2. user (2026-07-07T04:10:36.973Z)**

```
37:``--decode-backend nvdec_resident`` (default ``cpu``) moves decode + resize + normalize
66:# --stream-video); "nvdec_resident" = GPU-resident NVDEC decode + affine grid_sample
215:    decode_backend: str = "cpu",
223:    ``decode_backend`` follows the same pattern (#506): pre-existing caches (no field)
244:       and manifest.get("decode_backend", "cpu") == decode_backend
615:       decode_backend="cpu",  # run() IS the cpu decode path (disk frames / cv2 stream)
636:       "decode_backend": "cpu",
1004:                f"decode_backend=nvdec_resident needs PyNvVideoCodec with RGB output "
1006:                f"/ gpu_setup.sh stage them). Fall back to decode_backend=cpu."
1069:    ``decode_backend`` and never shared with cpu-decoded runs. Same resumable
1081:        raise SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")
1115:           decode_backend="nvdec_resident",
1136:           "decode_backend": "nvdec_resident",
1240:       "--frames_out_dir", help="where to extract frames when --video is used"
1243:       "--stream-video",
1249:       "--decode-backend",
1253:       "--stream-video). nvdec_resident = GPU-RESIDENT decode+resize+normalize "
1298:    if args.decode_backend == "nvdec_resident":
1300:        # --stream-video cv2 path nor the extract-frames fallback below may run.
1302:           raise SystemExit("--decode-backend nvdec_resident requires --video")
1305:               "--decode-backend nvdec_resident requires --device cuda "
1322:    if args.stream_video:
1324:           raise SystemExit("--stream-video requires --video")
1344:           args.video, args.frames_out_dir or (args.video + "_frames")
```

**3. user (2026-07-07T04:10:39.308Z)**

```
9:+    # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident" moves
decode+resize+normalize
18:+    decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
21:+    def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
25:+        if self.decode_backend == "nvdec_resident" and self.device not in ("cuda", "auto"):
27:+                f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda' or "
44:+_VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
55:+    # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
61:             stream_video=(True if sv is None else bool(sv)),
87:+        if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
89:+                f"decode_backend=nvdec_resident requires CUDA, but the effective device is "
99:-        if self.cfg.stream_video:
100:+        if self.cfg.decode_backend == "nvdec_resident":
103:+            # --stream-video nor --frames_out_dir applies and there is no frame cache to
clean.
105:+        elif self.cfg.stream_video:
113:+        if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
117:+                "decode_backend=nvdec_resident requires CUDA, but the effective device is "
138:+``--decode-backend nvdec_resident`` (default ``cpu``) moves decode + resize + normalize
152:+# --stream-video); "nvdec_resident" = GPU-resident NVDEC decode + affine grid_sample
154:+DECODE_BACKENDS = ("cpu", "nvdec_resident")
226:+    are ``cpu``-decoded, so they stay compatible with cpu runs ? but an ``nvdec_resident``
378:+    """On-GPU frame source for the ``nvdec_resident`` detector pass (#506).
401:+                f"decode_backend=nvdec_resident needs PyNvVideoCodec with RGB output "
446:+def run_video_nvdec_resident(
478:+        raise SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")
493:+            f"nvdec_resident: {total} frames ({fe.orig_w}x{fe.orig_h}), "
497:+        # ---- cache setup / resume decision (mirrors run(); keyed nvdec_resident) --- #
512:+            decode_backend="nvdec_resident",
533:+            "decode_backend": "nvdec_resident",
642:+        "--stream-video). nvdec_resident = GPU-RESIDENT decode+resize+normalize "
654:+    if args.decode_backend == "nvdec_resident":
658:+            raise SystemExit("--decode-backend nvdec_resident requires --video")
661:+                "--decode-backend nvdec_resident requires --device cuda "
664:+        run_video_nvdec_resident(
```

```
678:     if args.stream_video:
726:+# PyNvVideoCodec dlopens. Harmless when decode_backend=cpu; required for nvdec_resident.
744:+# not load) for decode_backend=nvdec_resident. wasb_infer.py carries the two version shims
this bump
781:+# nvdec_resident, ~1.7×). Step [2/6] stages the NVDEC/NVENC driver userspace libs
871:+except Exception as e:  # nvdec_resident unavailable; the cpu decode path still works
925:+    # inject neither) ? a silent drop of any of these bricks decode_backend=nvdec_resident.
954:+    idx = AppConfig(indexing={"wasb": {"decode_backend": "nvdec_resident"}}).indexing
955:+    assert NativeWasbConfig.from_indexing_cfg(idx).decode_backend == "nvdec_resident"
961:+    monkeypatch.setenv("WASB_DECODE_BACKEND", "nvdec_resident")
963:+    assert cfg.decode_backend == "nvdec_resident"
977:+        WasbIndexConfig(decode_backend="nvdec_resident", device="cpu")
981:+                "wasb": {"decode_backend": "nvdec_resident", "device": "mps"}
986:+        WasbIndexConfig(decode_backend="nvdec_resident").decode_backend
987:+        == "nvdec_resident"
990:+        WasbIndexConfig(decode_backend="nvdec_resident", device="auto").device
997:+    nvdec_resident until the full golden set passes at the SERVED preset ? a
1025:+def test_run_predict_nvdec_resident_threads_flag_and_skips_cv2_paths(tmp_path):
1026:+    """nvdec_resident decodes on the GPU inside wasb_infer.py: the flag is threaded and
1027:+    NEITHER cv2 path flag (--stream-video / --frames_out_dir) rides along; there is no
1030:+        NativeWasbConfig(weights_path="/m/w.pth", decode_backend="nvdec_resident"),
1034:+    assert argv[argv.index("--decode-backend") + 1] == "nvdec_resident"
1035:+    assert "--stream-video" not in argv and "--frames_out_dir" not in argv
1040:+def test_run_predict_nvdec_resident_auto_without_gpu_aborts_before_subprocess(tmp_path):
1045:+            weights_path="/m/w.pth", device="auto", decode_backend="nvdec_resident"
1065:+def test_healthcheck_nvdec_resident_requires_effective_cuda(tmp_path):
1066:+    """auto?cpu fallback is fine for the cpu decode path but NOT for nvdec_resident:
1069:+        _ok_cfg(tmp_path, device="auto", decode_backend="nvdec_resident")
1075:+    assert not ok and "nvdec_resident" in msg
1078:+def test_healthcheck_nvdec_resident_ok_with_cuda(tmp_path):
1080:+        _ok_cfg(tmp_path, device="cuda", decode_backend="nvdec_resident")
```

**4. user (2026-07-07T04:10:50.119Z)**

```
1230        return _tracker_pass_and_write(cfg, det_by_fid, out_csv)
1231
1232
1233    def main():
1234        ap = argparse.ArgumentParser()
1235        ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
1236        ap.add_argument(
1237            "--video", help="video file; frames are extracted internally (in WSL)"
1238        )
1239        ap.add_argument(
1240            "--frames_out_dir", help="where to extract frames when --video is used"
1241        )
1242        ap.add_argument(
1243            "--stream-video",
1244            action="store_true",
1245            help="FAST PATH: decode --video on the fly and feed frames straight to "
1246            "the detector (no PNG extraction). Output-equivalent to the disk path.",
1247        )
1248        ap.add_argument(
1249            "--decode-backend",
1250            default="cpu",
1251            choices=list(DECODE_BACKENDS),
1252            help="cpu (default) = the historical cv2 decode paths (disk frames or "
1253            "--stream-video). nvdec_resident = GPU-RESIDENT decode+resize+normalize "
1254            "(#506): PyNvVideoCodec NVDEC + cv2-faithful affine grid_sample, CUDA-only, "
1255            "~1.7x on an L4. Requires --video; detections cache separately from cpu "
1256            "(trajectory-level, not byte-level, equivalence).",
1257        )
1258        ap.add_argument("--weights", required=True)
1259        ap.add_argument("--out", required=True)
1260        ap.add_argument("--sport", default="badminton")
```

```
        ap.add_argument(
            "--device",
            default="cuda",
            choices=["cuda", "mps", "cpu"],
            help="torch device for the detector (default cuda = the WSL parity path)",
        )
        ap.add_argument(
            "--limit", type=int, default=0, help="cap #frames (for quick tests)"
        )
        ap.add_argument("--batch-size", type=int, default=8)
        ap.add_argument("--num-workers", type=int, default=4)
        ap.add_argument(
            "--prefetch-batches",
            type=int,
            default=0,
            help="streaming mode: overlap CPU batch assembly with GPU inference via a "
            "bounded producer thread of this queue depth (0 = off, the parity default)",
        )
        ap.add_argument(
            "--log-every-batches", type=int, default=50, help="progress log cadence"
        )
        ap.add_argument(
            "--cache-dir",
            default=None,
            help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
        )
        ap.add_argument(
            "--flush-every",
            type=int,
            default=1000,
            help="fsync the detector cache every N windows (bounds reboot loss)",
        )
        ap.add_argument(
            "--fresh", action="store_true", help="ignore any existing cache and start over"
        )
        args = ap.parse_args()

        if args.decode_backend == "nvdec_resident":
            # GPU-resident branch FIRST: it decodes the video itself (NVDEC), so neither the
            # --stream-video cv2 path nor the extract-frames fallback below may run.
            if not args.video:
                raise SystemExit("--decode-backend nvdec_resident requires --video")
            if args.device != "cuda":
                raise SystemExit(
                    "--decode-backend nvdec_resident requires --device cuda "
                    "(NVDEC decode runs on the GPU)"
                )
            run_video_nvdec_resident(
                args.video,
                args.weights,
                args.out,
                args.sport,
                args.limit,
                batch_size=args.batch_size,
                log_every_batches=args.log_every_batches,
                cache_dir=args.cache_dir,
                flush_every=args.flush_every,
                fresh=args.fresh,
            )
            return

        if args.stream_video:
            if not args.video:
                raise SystemExit("--stream-video requires --video")
            run_video_streaming(
```

```
                args.video,
                args.weights,
                args.out,
                args.sport,
                args.limit,
                batch_size=args.batch_size,
                log_every_batches=args.log_every_batches,
                cache_dir=args.cache_dir,
                flush_every=args.flush_every,
                fresh=args.fresh,
                device=args.device,
                prefetch_batches=args.prefetch_batches,
            )
            return

        frames_dir = args.frames_dir
        if args.video:
            frames_dir = extract_frames(
                args.video, args.frames_out_dir or (args.video + "_frames")
            )
        if not frames_dir:
            raise SystemExit("provide --frames_dir or --video")
        run(
            frames_dir,
            args.weights,
            args.out,
            args.sport,
            args.limit,
            batch_size=args.batch_size,
            num_workers=args.num_workers,
            log_every_batches=args.log_every_batches,
            cache_dir=args.cache_dir,
            flush_every=args.flush_every,
            fresh=args.fresh,
            device=args.device,
        )


    if __name__ == "__main__":
        main()

```

**5. user (2026-07-07T04:11:42.930Z)**

```
    ) -> str:
        """GPU-resident path (#506): NVDEC-decode ``video_path`` on the GPU and feed
        warped+normalized frames straight to the detector ? decode, resize, and
        normalize never touch the host. CUDA-only by construction.

        NOT byte-identical to the ``cpu`` decode paths (NVDEC's YUV?RGB and the GPU
        warp differ from cv2 at float precision) ? equivalence is trajectory-level
        (the #506 unit gate: 96.3% visibility agreement, median 0.036 px) and
        rally-F1-level (the golden gate), so the detection cache is keyed by
        ``decode_backend`` and never shared with cpu-decoded runs. Same resumable
        cache + tracker tail as ``run()`` otherwise.
        """
        import time

        import numpy as np
        import torch

        from detectors import build_detector

        _ensure_np_inf()
        if not torch.cuda.is_available():
```

```
            raise SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")

        cfg = _build_cfg(weights, sport, "cuda")
        frames_in = int(cfg["model"]["frames_in"])
        input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
        output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
        out_scales = list(cfg["model"]["out_scales"])

        fe = NvdecResidentFrontend(video_path, input_wh)
        try:
            total = min(fe.count, limit) if limit else fe.count
            if total < frames_in:
                raise SystemExit(f"need >= {frames_in} frames, video has {total}")
            windows_total = total - frames_in + 1
            logger.info(
                f"nvdec_resident: {total} frames ({fe.orig_w}x{fe.orig_h}), "
                f"{windows_total} windows"
            )

            # ---- cache setup / resume decision (mirrors run(); keyed nvdec_resident) --- #
            if cache_dir is None:
                cache_dir = default_cache_dir(out_csv)
            os.makedirs(cache_dir, exist_ok=True)
            det_jsonl, _ = _cache_paths(cache_dir)
            cache_key = "video:" + osp.abspath(video_path)
            manifest = None if fresh else read_manifest(cache_dir)
            resume = manifest_compatible(
                manifest or {},
                frames_dir=cache_key,
                weights=weights,
                sport=sport,
                frames_in=frames_in,
                frames_total=total,
                device="cuda",
                decode_backend="nvdec_resident",
            )
            if resume:
                windows_done, det_by_fid = load_and_clean_cache(cache_dir)
                logger.info(
                    f"resuming from cache: {windows_done}/{windows_total} windows already
detected"
                )
            else:
                if manifest is not None:
                    logger.info("cache incompatible with this run -> starting fresh")
                reset_cache(cache_dir)
                windows_done, det_by_fid = 0, defaultdict(list)

            base_manifest = {
                "version": CACHE_VERSION,
                "frames_dir": cache_key,
                "weights": weights,
                "sport": sport,
                "frames_in": frames_in,
                "frames_total": total,
                "device": "cuda",
                "decode_backend": "nvdec_resident",
                "windows_total": windows_total,
                "windows_done": windows_done,
            }
            write_manifest(cache_dir, base_manifest)

            # ---- GPU-resident detector pass over the un-cached windows ----------------- #
            if windows_done < windows_total:
                with _torch_load_compat():
```

```
1145                     detector = build_detector(cfg)
1146                 # {scale: [1,2,3] float64 CPU tensor} ? the postprocessor .cpu().numpy()'s
1147                 # these, so CPU float64 matches the DataLoader-collated path exactly.
1148                 trans_t = {
1149                     s: torch.from_numpy(np.ascontiguousarray(m)).unsqueeze(0)
```

**6. user (2026-07-07T04:12:25.978Z)**

```
Structured output provided successfully
```